EX16 - Trajectory Processing - Boolean Masks

Beginner Step-by-Step Guide

This guide follows the real file `ex16_trajectory_data_processing.py`. It explains the problem first and then the code from the perspective of someone starting Python.

1. What we want to solve

Answer trajectory questions with vectorized conditions: ascent, descent, altitude, time windows, statistics, indices, energy and dt.

Problem variables / elements

Name	Meaning
mask	True/False array
processed_data	matrix [h,v,t,energy]

Equation or logic on paper

```
mask = condition on a column
filtered = A[mask]
dt_k = t[k+1] - t[k]
```

Manual check

If velocities=[2,-1,3], velocities>0 gives [True,False,True]. Applying that mask to the dataset keeps rows 0 and 2.

2. What we already know and what is new

Already seen:

- arrays
- column slicing
- vectorization

New in this exercise:

- Boolean mask
- A[mask]
- &
- np.sum(mask)
- np.where
- mean/std
- column_stack
- diff
- allclose

3. New Python concepts

1. Mask

Syntax: `ascending_mask = A[:,1] > 0`

The comparison applies to each element and returns True/False per sample.

2. Filtering

Syntax: `A[mask]`

NumPy keeps rows whose corresponding mask position is True.

3. Vector AND

Syntax: `(times >= 2) & (times <= 5)`

With arrays we use & to require both conditions element-wise.

4. Indices

Syntax: `np.where(mask)[0]`

Returns positions where the mask is True.

5. Differences

Syntax: `np.diff(times)`

Computes consecutive differences, so N samples produce N-1 intervals.

4. Code step by step

Step 1. Create masks

```
ascending_mask = trajectory_array[:, 1] > 0
descending_mask = trajectory_array[:, 1] < 0
```

First select velocity and then compare all its values.

Step 2. Filter states

```
ascending_states = trajectory_array[ascending_mask]
```

Applying the mask to the full matrix returns [h,v,t] rows.

Step 3. Combine conditions

```
time_window = ((trajectory_array[:,2] >= 2) &
               (trajectory_array[:,2] <= 5))
```

Each comparison generates a mask; & requires True in both.

Step 4. Statistics

```
mean_height = np.mean(trajectory_array[:, 0])
std_height = np.std(trajectory_array[:, 0])
```

The functions receive a numeric column and return scalars.

Step 5. Add energy

```
processed_data = np.column_stack((trajectory_array, energy))
```

energy has one value per row and is added as the fourth column.

Step 6. Check dt

```
dt_samples_diff = np.diff(trajectory_array[:, 2])
dt_samples_allclose = np.allclose(dt_samples_diff, config["dt"])
```

diff gets all intervals and allclose checks if they are close to dt within tolerance.

5. Execution flow

```
trajectory matrix
 -> masks
 -> filtered rows / statistics / indices
 -> energy column
 -> processed_data
 -> dt check
```

6. Expected result / check

The dt check should report Time Matches and processed_data should have four columns.

7. Common beginner mistakes

- Using and instead of & with arrays.
- Forgetting parentheses in compound conditions.
- Confusing np.sum(mask) with indices.
- Expecting N values from np.diff for N samples.

8. If you remember only five things

- A mask is a Boolean array.
- A[mask] filters data.
- & combines conditions.
- np.where locates positions.
- np.diff and allclose help validate sampling.

9. Mini-practice

Change one input. Before running, write a comment predicting what should change and why. Then run the file and compare your prediction with the output.

Completion criterion

Move on when you can explain the problem, equation/logic, new Python concepts and execution flow without simply reading the code.